

Grundlagen der Informatik

Zahlensysteme / Elementare Datentypen

Ferdinand Pöll, Tobias Walter

Hochschule Mainz University of Applied Sciences

ferdinand.poell@lba.hs-mainz.de, tobias.walter@hs-mainz.de

5. Oktober 2025

- 1 Unterschiedliche Zahlensysteme
- 2 Dual-, Oktal- und Hexadezimalsystem
- 3 Konvertierungsalgorithmen
- 4 Rechenoperationen im Dualsystem
- 5 Reelle Zahlen
- 6 Duale Größenangaben
- 7 Grunddatentypen
- 8 Literatur

- **Additive Zahlensysteme** Der Wert einer Zahl wird durch Addieren der Werte ihrer Ziffern errechnet.
- **Hybriden Zahlensysteme** Hierbei wird eine Grundziffer einem Zeichen vorangestellt, das eine Potenz der Basis wiedergibt; die Werte beider werden miteinander multipliziert. In den europäischen Zahlensystemen kamen solche Hybridsysteme so gut wie nicht vor, wohl aber, schon seit Beginn des zweiten Jahrtausends v. Chr., in Mesopotamien, später auch in China und im Nahen Osten allgemein. Sowohl aus Äthiopien als auch aus Südindien und Sri Lanka sowie der Maya-Kultur sind solche hybriden Zahlensysteme bekannt.
[Wiki:Zahlensystem,2017]
- **Positionellen Zahlensysteme** Die (additive) Wertigkeit eines Symbols hängt von seiner Position ab.

Additives Zahlensystem – Römische Zahlen

- Verfügbare Ziffern (bzw. unser Alphabet) mit Entsprechung:
 $1 = I$, $5 = V$, $10 = X$, $50 = L$, $100 = C$, $500 = D$, $1000 = M$
- Die Ziffern I , X , C und M dürfen beliebig oft nebeneinander stehen
- Die Ziffern V , L und D dürfen nicht wiederholt nebeneinander stehen.
- Die Ziffern I , X und C dürfen nicht mehr als dreimal nebeneinander stehen.
- M kann beliebig oft hintereinander stehen.
- V , L und D dürfen in einer Zahl nur einmal vorkommen.
- Kleinere Einheit rechts einer Zahl werden addiert.
- Kleinere Einheit links einer Zahl wird subtrahiert.
- Es dürfen nicht zwei oder mehrere kleine Einheiten von der rechts stehenden größeren Einheit abgezogen werden.

Additives Zahlensystem – Römische Zahlen

VI

DCCLVI

IV

IX

XXIX

Additives Zahlensystem – Römische Zahlen

$$\text{VI} \quad \equiv 6 = 5 + 1$$

$$\text{DCCLVI} \quad \equiv 756 = 500 + 100 + 50 + 5 + 1$$

$$\text{IV} \quad \equiv 4 = 5 - 1$$

$$\text{IX} \quad \equiv 9 = 10 - 1$$

$$\text{XXIX} \quad \equiv 29 = 10 + 10 + 10 - 1$$

- Zahlensysteme wurden in der Vergangenheit sehr unterschiedlich konzipiert.
- These: Fast alle Zahlensysteme beruhen auf dem Abzählen mit den Fingern. Es findet sich daher fast überall in mehr oder weniger unterschiedlichen Varianten das Zehnersystem.
[Wohlrade et al., 2012]

Positionssysteme

- Zahlensysteme wurden in der Vergangenheit sehr unterschiedlich konzipiert.
- These: Fast alle Zahlensysteme beruhen auf dem Abzählen mit den Fingern. Es findet sich daher fast überall in mehr oder weniger unterschiedlichen Varianten das Zehnersystem.
[Wohlrath et al., 2012]

 1	 11	 21	 31	 41	 51
 2	 12	 22	 32	 42	 52
 3	 13	 23	 33	 43	 53
 4	 14	 24	 34	 44	 54
 5	 15	 25	 35	 45	 55
 6	 16	 26	 36	 46	 56
 7	 17	 27	 37	 47	 57
 8	 18	 28	 38	 48	 58
 9	 19	 29	 39	 49	 59

Positionssysteme bei natürlichen Zahlen

Eine Zahl n wird in einem Positionssystem zur Basis B nach seinen Potenzen zerlegt dargestellt.

$$n = \sum_{i=0}^{N-1} b_i * B^i$$

- n natürliche Zahl
- B Basis des Zahlensystems ($B \in \mathbb{N}, B \geq 2$)
- b Ziffern ($b_i \in \mathbb{N}_0, 0 \leq b_i \leq B$)
- N Anzahl der Stellen

Positionssysteme bei natürlichen Zahlen

Beispiele für Zahlensysteme

$B = 2$ Dualsystem

$B = 8$ Oktalsystem

$B = 10$ Dezimalsystem

$B = 16$ Hexadezimalsystem

Positionssysteme bei natürlichen Zahlen

Dual	Oktal	Dezimal	Hexadezimal
0	0	0	0
1	1	1	1
10	2	2	2
11	3	3	3
100	4	4	4
101	5	5	5
110	6	6	6
111	7	7	7
1000	10	8	8
1001	11	9	9
1010	12	10	A
1011	13	11	B
1100	14	12	C
1101	15	13	D
1110	16	14	E

Positionssysteme bei gebrochenen Zahlen

Bei gebrochenen Zahlen trennt ein Punkt (bzw. in Deutschland ein Komma) in der Zahl den ganzzahligen Teil vom Nachkommaanteil.

Positionssysteme bei gebrochenen Zahlen

Bei gebrochenen Zahlen trennt ein Punkt (bzw. in Deutschland ein Komma) in der Zahl den ganzzahligen Teil vom Nachkommaanteil.

$$n = \sum_{i=-M}^{N-1} b_i * B^i$$

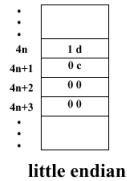
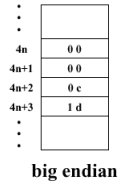
- n reelle Zahl
- B Basis des Zahlensystems ($B \in \mathbb{N}, B \geq 2$)
- b Ziffern ($b_i \in \mathbb{N}_0, 0 \leq b_i \leq B$)
- N Anzahl der Stellen vor dem Komma
- M Anzahl der Stellen hinter dem Komma

- Positionssysteme mit einem $B > 2$ sind technisch schwer zu realisieren.
- In der Praxis wird zumeist eine binäre Repräsentation gewählt.
- Eine einzelne Binärstelle wird als Bit (Binary Digit) bezeichnet.

Endianness

- Steigt bei einer im Speicher abgelegten Zahl die Wertigkeit einer Stelle mit der wachsenden Adresse, dann ist sie im *Little-Endian-Format* dargestellt.
- Fällt bei einer im Speicher abgelegten Zahl die Wertigkeit einer Stelle mit der wachsenden Adresse, dann ist sie im *Big-Endian-Format* dargestellt.

$$(3101)_{10} = 12 * 16^2 + 1 * 16^1 + 13 * 16^0$$
$$= (00\ 00\ 0c\ 1d)_{16}$$



Big Endian speichert den höherwertigen Teil (00) am oberen oder Anfang der Adresse, und Little Endian ist die Umkehrung von Big Endian (speichert (1d) am Anfang der Adresse).

Endianness Beispiel

Im Beispiel wird die Ganzzahl 16.909.060 als 32-Bit-Integer-Wert gespeichert (hexadezimal: 01020304h). Die Speicherung erfolgt in 4 Bytes ab einer angenommenen Speicheradresse von 10000.

- Big-Endian speichert die Beispielzahl in der Reihenfolge 01 02 03 04 = 01h02h03h04h
- Little-Endian speichert die Bytes in der umgekehrten Reihenfolge 04 03 02 01 = 04h03h02h01h
- Einige ältere Systeme (z. B. PDP-11) speichern die Bytes auch in der Reihenfolge 02 01 04 03 = 02h01h04h03h. Dies wird als Mixed-Endian oder auch als Middle-Endian bezeichnet.

Adresse	Big	Little	Mixed
10000	01	04	02
10001	02	03	01
10002	03	02	04
10003	04	01	03

Konvertieren von Zahlen in das Dezimalsystem

Zu einem Polynom $p(x) = b_0 + b_1x + b_2x^2 + \dots + b_nx^n$ vom Grad n ist das Horner-Schema definiert als:

$$p(x) = (\dots (b_nx + b_{n-1})x + \dots)x + b_0$$

Eine Zahl im Positionssystem ist definiert als $n = \sum_{i=0}^{N-1} b_i * B^i$, was es uns erlaubt, dass Horner-Schema anzuwenden.

Beispiel:

$$(1578)_{10} = ((1 * 10 + 5) * 10 + 7) * 10 + 8$$

$$(754)_8 = (7 * 8 + 5) * 8 + 4$$

Konvertieren von Zahlen in das Dezimalsystem

$(375)_8$

$(1210)_8$

$(888)_9$

$(ADA)_{16}$

Konvertieren von Dezimalzahlen in andere Systeme

Für die Umwandlung einer Dezimalzahl n in ein Zahlensystem mit der Basis B kann folgender Algorithmus verwendet werden:

- 1 $n : B = y$ Rest z
- 2 $n := y$, Schritt 1, wenn $n \neq 0$ sonst Schritt 3
- 3 Die ermittelten Reste entsprechen der Zahl zur Basis B .

Konvertieren von Dezimalzahlen in andere Systeme

$(445)_{10}$ in das Dualsystem

$(7294)_{10}$ in das Oktalsystem

$(87599)_{10}$ in das Hexadezimalsystem

$(754498)_{10}$ in das Dualsystem

Konvertieren von Dezimalzahlen in andere Systeme

$(445)_{10}$ in das Dualsystem

$(7294)_{10}$ in das Oktalsystem

$(87599)_{10}$ in das Hexadezimalsystem

$(754498)_{10}$ in das Dualsystem

Kann man diese letzte Zahl eventuell auch effizienter konvertieren?

Konvertieren echt gebrochener Zahlen

Eine echt gebrochene Zahl $n(n < 1)$:

$$n = \sum_{i=-M}^{-1} b_i * B^i$$

lässt sich nach Horner-Schema wie folgt darstellen:

$$n = \frac{1}{B} * (b_{-1} + \frac{1}{B}(b_{-2} + \frac{1}{B}(b_{-3} + \dots + \frac{1}{B}(b_{-M+1} + \frac{1}{B}b_{-M}) \dots))$$

wie z. B. die Zahl

$$0.123 = \frac{1}{10}(1 + \frac{1}{10}(2 + \frac{1}{10}3))$$

Konvertieren echt gebrochener Zahlen

Algorithmus zur Umwandlung:

- 1 $n * B = y$ Überlauf z
- 2 $n := y$, Schritt 1, wenn $n \neq 0$ sonst Schritt 3
- 3 Die ermittelten Überläufe ergeben die entsprechende Zahl zur Basis B .

Beispiel: $(0.34375)_{10} = (0.01011)_2$

n	y	z
0.34375 * 2	0.6875	0
0.6875 * 2	1.375	1
0.375 * 2	0.75	0
0.75 * 2	1.5	1
0.5 * 2	1	1
0 * 2	0	0

Boolsches Rechnen – Boolesche Addition

Regeln:

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 0 \text{ Übertrag } 1$$

$$1 + 1 + 1 = 1 \text{ Übertrag } 1$$

$$\begin{array}{r} \bullet 5_{\text{ten}} + 6_{\text{ten}} \\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0101\ (5_{\text{ten}}) \\ + 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0110\ (6_{\text{ten}}) \\ \hline = 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1011\ (11_{\text{ten}}) \end{array}$$

... 0 0 1 0 1
+ ... 0 0 1 1 0
... 0 (0)1 (1)0 (0)1 (0)1

Carries

Boolsches Rechnen – Darstellung negativer Zahlen

Alle Kombinationen, bei denen das 1. Bit (Vorzeichenbit) gesetzt ist, repräsentieren dabei negative Zahlen:

0000 = 0		1000 = -8
0001 = 1		1001 = -7
0010 = 2		1010 = -6
0011 = 3		1011 = -5
0100 = 4		1100 = -4
0101 = 5		1101 = -3
0110 = 6		1110 = -2
0111 = 7		1111 = -1

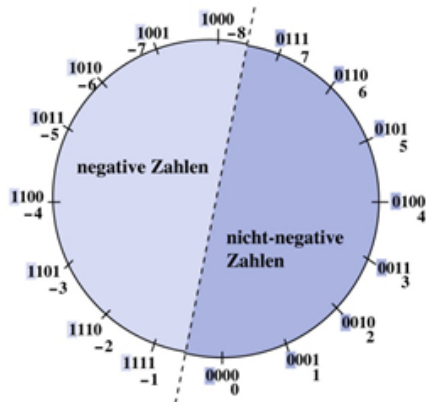


Abbildung 3.2: Zahlenring für vier Bits, wobei das erste Bit das Vorzeichenbit ist

- Most Significant Bit (MSB) repräsentiert Vorzeichen (1 $\equiv$ negativ)
- Null ist definiert als positiv

Rechnen auf Binär - B Komplement und Subtraktion

- Jedes Bit wird invertiert.
- Auf den invertierten Wert wird 1 addiert.
- Zur Subtraktion wird vom Minuend das Komplement subtrahiert.

$$\bullet 12_{\text{ten}} - 5_{\text{ten}}$$

$$\begin{array}{r} 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1100\ (12_{\text{ten}}) \\ - 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0101\ (5_{\text{ten}}) \\ \hline = 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0111\ (7_{\text{ten}}) \end{array}$$

$$\bullet 12_{\text{ten}} - 5_{\text{ten}} = 12_{\text{ten}} + (-5_{\text{ten}})$$

$$\begin{array}{r} 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1100\ (12_{\text{ten}}) \\ + 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1011\ (-5_{\text{ten}}) \\ \hline = 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0111\ (7_{\text{ten}}) \end{array}$$

Ein Überlauf tritt auf, wenn eine Binärzahldarstellung nicht genügend Bits enthält, um das Ergebnis einer arithmetischen Operation darzustellen. Ein Überlauf tritt auf, weil die Computerarithmetik in Bezug auf Addition, Subtraktion, Multiplikation oder Division nicht geschlossen ist. Ein Überlauf kann nicht zusätzlich auftreten (Subtraktion), wenn die Operanden unterschiedliche (bzw. identische) Vorzeichen haben.

$$\begin{array}{r} 7 + 6 \\ 0111 \quad (7_{\text{ten}}) \\ + 0110 \quad (6_{\text{ten}}) \\ \hline 1101 \quad (13_{\text{ten}}) \end{array}$$

$$\begin{array}{r} -7 + -6 \\ 1001 \quad (-7_{\text{ten}}) \\ + 1010 \quad (-6_{\text{ten}}) \\ \hline 0011 \quad (-13_{\text{ten}}) \end{array}$$

$$\begin{array}{r} -7 - 6 \\ 1001 \quad (-7_{\text{ten}}) \\ - 0110 \quad (6_{\text{ten}}) \\ \hline 0011 \quad (-13_{\text{ten}}) \end{array}$$

$$\begin{array}{r} -7 - 6 = -7 + -6 \\ 1001 \quad (-7_{\text{ten}}) \\ + 1010 \quad (-6_{\text{ten}}) \\ \hline 0011 \quad (-13_{\text{ten}}) \end{array}$$

Um einen Überlauf zu erkennen und zu kompensieren, benötigt man $n + 1$ Bits, wenn eine n -Bit-Zahldarstellung verwendet wird.

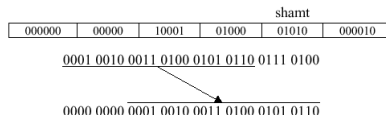
Multiplikation wird realisiert durch folgende Verfahren:

- Wiederholte Addition
- Analog dem gewohnten dezimalen System, und kann in digitalen Schaltungen als eine Abfolge von Additionen und Schiebeoperationen realisiert werden
- Parallele Multiplikation

Rechnen auf Binär - Multiplikation

Um eine binäre Zahl effizient nach links oder rechts zu verschieben, werden Schiebeoperationen verwendet.

- sll (logisch nach links verschieben): Füllt leere Bits mit Nullen
- srl (logisch nach rechts verschieben): Füllt leere Bits mit Nullen
- sra (Arithmetik nach rechts verschieben): Vorzeichen erweitert leere Bits



Beispiel. Angenommen, die Anweisung im R-Format `srl $t0, $s1, 8` wird wie unten gezeigt angewendet. Der Verschiebungsbetrag, dargestellt als *shamt*, ist die Anzahl der Bits, um den Operanden `$s1` zu verschieben, um das Ergebnis zu erhalten `$t0`.

- Festkommazahlen – Das Komma steht immer an einer festgelegten Stelle.
 - Man kann nur einen beschränkten Wertebereich darstellen.
 - Stelle des Punktes festgelegt, daher sehr wenig flexibel.
 - Einsatz in Spezialbereichen.
- Gleitkommazahlen – Mantisse und Exponent werden gespeichert.

Eine Möglichkeit zur Darstellung von reellen Zahlen im Binärsystem besteht darin, Binärzahlen mit einer festen Anzahl von Ziffern zu verwenden und festzulegen, ab welcher Stelle der Vorkommateil endet und der Nachkommateil beginnt. Da das (gedachte) Komma hier an einer festen Stelle ist, werden Zahlen, die in dieser Form dargestellt werden, Festkommazahlen genannt.

Je mehr Ziffern für die Vorkommastellen reserviert sind, desto größer ist der Wertebereich. Umgekehrt bedeuten weniger Ziffern für die Vorkommastellen mehr Ziffern für die Nachkommastellen und damit eine höhere Genauigkeit.

Bei der Nutzung von Festkommazahlen muss man sich also zwischen großem Wertebereich und großer Genauigkeit entscheiden oder einen Kompromiss wählen.

Reelle Zahlen - Gleitkommazahlen

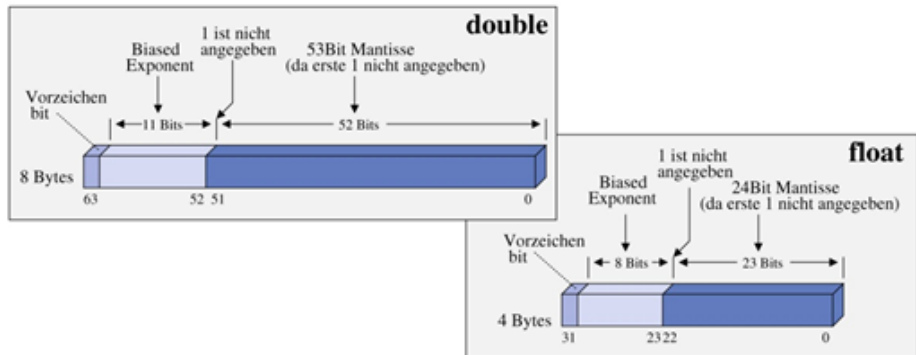


Abbildung 3.4: IEEE-Format für float und double

IEEE 754 definiert normalisierte Gleitpunktzahlen. Hierbei ist das Komma stets hinter der führenden 1. Die führende 1 wird nicht gespeichert.

Exponent	Mantisse	Bedeutung
111...111	$\neq 0$	Not a number

Gleitkommazahlen

Die Basis muss nicht gespeichert werden, da Computer stets die Basis $b=2$ verwenden. Die Vorkommateil der Mantisse beträgt immer 1, da er im Intervall $[1; b[$ liegen muss.

Sollen sowohl positive wie auch negative Zahlen gespeichert werden, ist in der Regel das erste Bit zur Abbildung des Vorzeichens reserviert. Eine 1 bedeutet dabei, dass es sich um eine negative Zahl handelt.

Desweiteren muss vorgegeben sein, wieviele der verbleibenden Bit für den Exponenten reserviert sind. Die restlichen Bit sind dann für die Speicherung der Mantisse verfügbar.

Beispiel: Es gilt, dass $z = m * b^e$, wobei $m \in [1; b[$

Dezimal	Gleitkomma (Basis $b=10$)	Gleitkomma (Basis $b=2$)
z	$m * 10^e$	$m * 2^e$
0,1865	$1,865 * 10^{-1}$	$1,492 * 2^{-3}$
18,65	$1,865 * 10^1$	$1,165625 * 2^4$
186,5	$1,865 * 10^2$	$1,45703125 * 2^7$

Tabelle 3.3

Maßeinheiten für Bytes

Maßeinheit		Anzahl von Bytes	KBytes	MBytes
Byte		1		
Kilobyte (KByte)	2^{10}	1024	1	
Megabyte (MByte)	2^{20}	1.048.576	1024	1
Gigabyte (GByte)	2^{30}	1.073.741.824	1.048.576	1024
Terabyte (TByte)	2^{40}	1.099.511.627.776	1.073.741.824	1.048.576
Petabyte (PByte)	2^{50}	1.125.899.906.842.624	1.099.511.627.776	1.073.741.824
Exabyte (EByte)	2^{60}	1.152.921.504.606.846.976	1.125.899.906.842.624	1.099.511.627.776

Tabelle 3.4

Typische Wertebereiche für die Datentypen auf 32-Bit-Architekturen

Datentyp-Bezeichnung	Bitzahl	Wertebereich
char, signed char	8	-128...127
unsigned char	8	0...255
short, signed short	16	-32 768...32 767
unsigned short	16	0...65 535
int, signed int	32	-2 147 483 648...2 147 483 647
unsigned, unsigned int	32	0...4 294 967 295
long, signed long	32	-2 147 83 648...2 147 483 647
unsigned long	32	0...4 294 967 295
float	32	$1.2 \cdot 10^{-38} \dots 3.4 \cdot 10^{38}$
double	64	$2.2 \cdot 10^{-308} \dots 1.8 \cdot 10^{308}$
long double	96	$3.4 \cdot 10^{-4932} \dots 1.1 \cdot 10^{4932}$

C++ definiert ein Byte folgendermaßen (Siehe [Datatypes])

- Memory is laid out as a sequence of something C++ calls "bytes".
- Each of these things that the C++ language calls a byte has at least 8 bits, but might have more than 8 bits.
- Guarantees that a `char*` (char pointers) can address individual bytes.
- There are no bits between two bytes. This means every bit in memory is part of a byte. If you grind your way through memory via a `char*`, you will be able to see every bit.
- There are no bits that are part of two distinct bytes. This means a change to one byte will never cause a change to a different byte.
- Gives you a way to find out how many bits are in a byte in your particular implementation: include the header `<climits>`, then the actual number of bits per byte will be given by the `CHAR_BIT` macro.



HEROLD, Helmut; LURZ, Bruno; WOHLRAB, Jürgen.

Grundlagen der Informatik.

Pearson Studium, 2012.



ERTEL, Wolfgang; BUßMANN, Paul; GLATZ, Karl.

Grundlagen der Informatik.

<https://www.hs-weingarten.de/~ertel/vorlesungen/ginf/skript-bussmann.pdf>



Wikipedia.

Zahlensystem.

<https://de.wikipedia.org/wiki/Zahlensystem>



Wikipedia.

Byte-Reihenfolge.

<https://de.wikipedia.org/wiki/Byte-Reihenfolge>



Wikipedia.

Horner-Schema.

<https://de.wikipedia.org/wiki/Horner-Schema>



Standard C++ Foundation.

Built-in / Intrinsic / Primitive Data Types.

<https://isocpp.org/wiki/faq/intrinsic-types#bytes-review>

The End